

Erwan KAAWAR

Ivan LAI

Tommy LIM



Projet Base de données

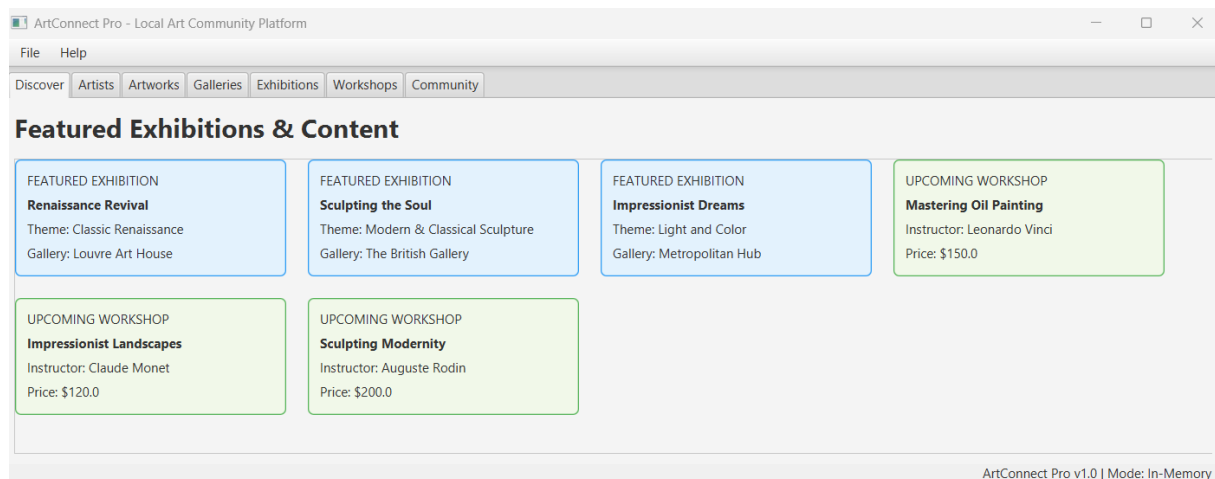
Table des matières

Étape 1 : Comprendre ArtConnect et définir le périmètre fonctionnel	3
Étape 2: Modélisation conceptuelle et logique de la base ArtConnect	6
Étape 3 : Implémentation de la Base de Données et Fonctionnalités Avancées	10
Étape 4: Intégration de la base dans l'application Java ArtConnect.....	13

Étape 1 : Comprendre ArtConnect et définir le périmètre fonctionnel

1. Exploration de l'application fournie

Importation et lancement de l'application :



On peut identifier comme écrans principaux, les pages suivantes : Discover, Artists, Artworks, Galleries, Exhibitions, Workshops, Community

2. Analyse fonctionnelle

Listez les fonctionnalités principales que vous voyez dans l'interface (du point de vue utilisateur).

Du point de vue utilisateur, plusieurs fonctionnalités sont disponibles. Tout d'abord, il y a la possibilité de filtrer les titres, tels que le nom, la ville, l'email par ordre alphabétique, et ceux avec des nombres comme les prix, la date de naissance par ordre de grandeur. Puis il y a également une fonctionnalité pour rechercher des artistes par leur nom, pour visualiser plus rapidement les artistes qui nous intéressent.



Identifiez au moins deux rôles ou profils d'utilisation (par exemple : “organisateur” vs “visiteur”; même si l'interface actuelle ne gère pas encore l'authentification, vous pouvez les définir conceptuellement).

Il semblerait qu'il y ait 3 différents profils possible dans le cadre de notre projet. Celui de l'administrateur, celui des organisateurs, et celui du visiteur.

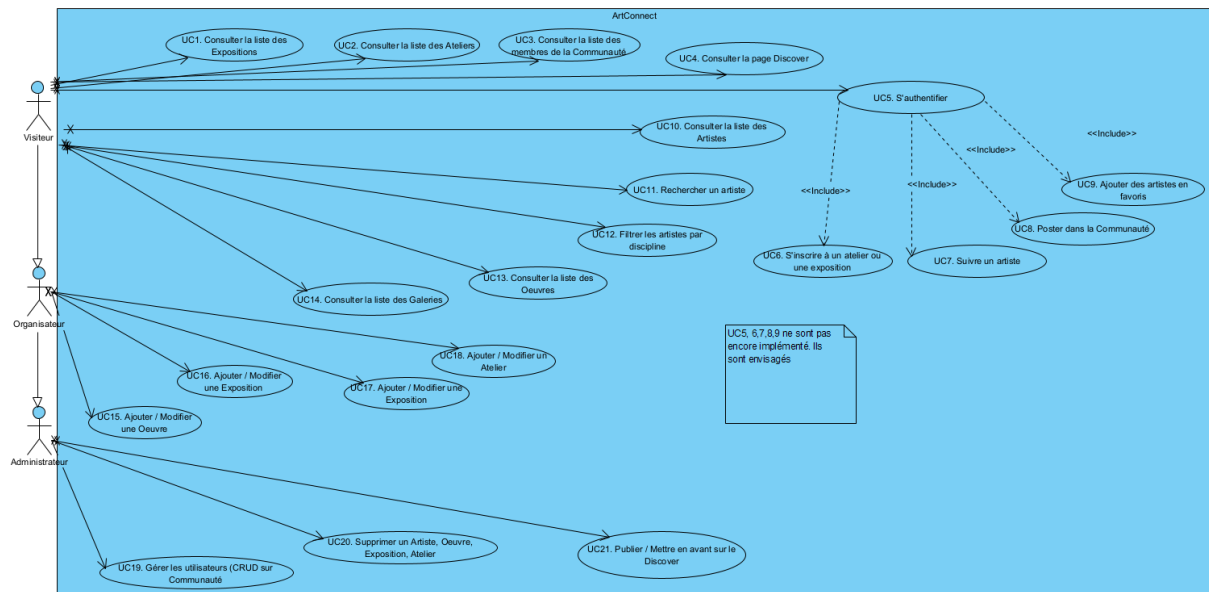
L'administrateur aurait accès à toutes les fonctionnalités. Il aurait la possibilité de lecture, d'ajout, de suppression, de gestion des utilisateurs, de filtrage et de recherche dans la base de données.

Le visiteur lui aurait accès seulement à la lecture, au filtrage et à la recherche.

L'organisateur aurait les mêmes droits que le visiteur mais avec la possibilité d'ajouter des données.

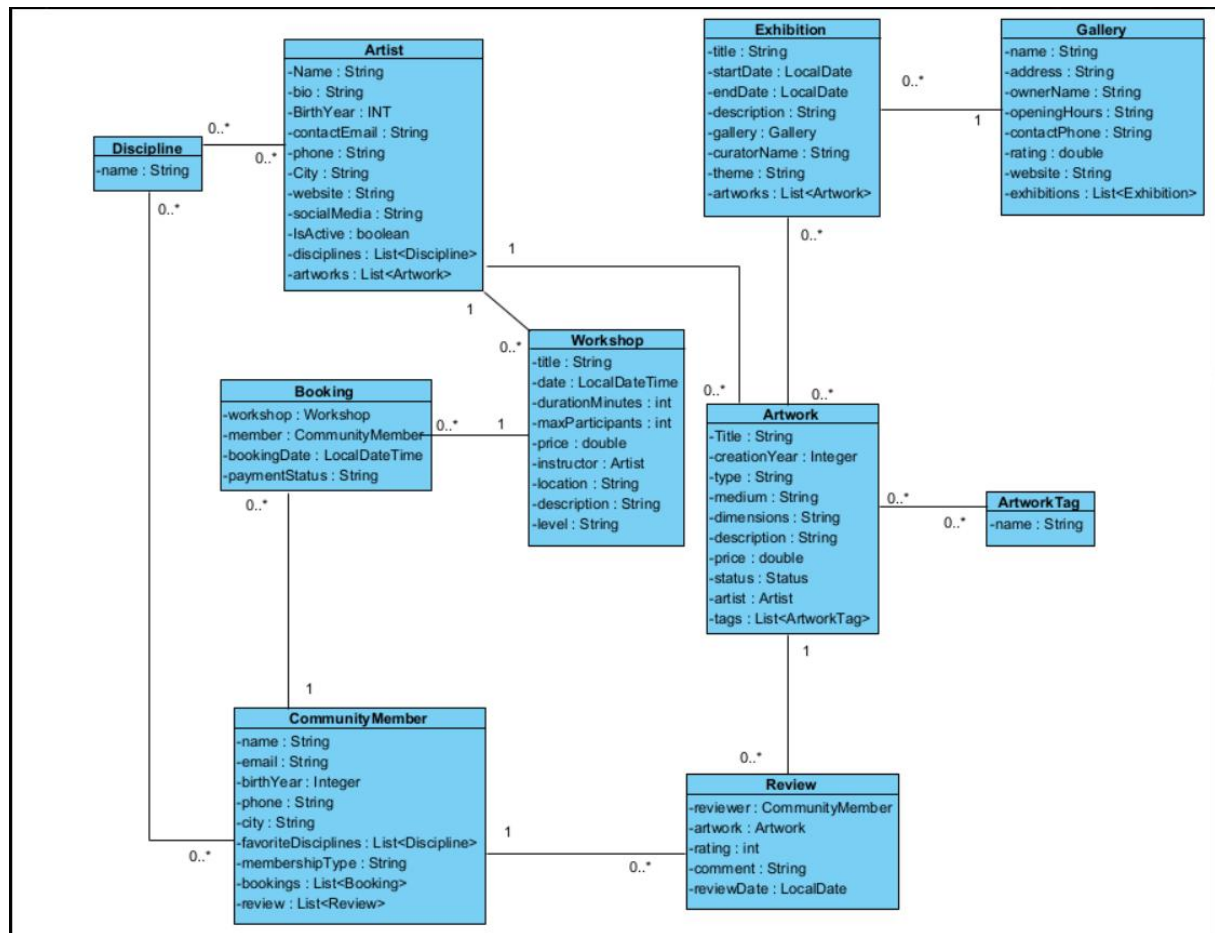
Dessinez un diagramme de cas d'utilisation (UML) représentant les fonctionnalités cibles d'ArtConnect du point de vue des acteurs.

Ajoutez une légende indiquant quelles fonctionnalités sont déjà présentes dans l'interface et lesquelles sont envisagées pour plus tard (optionnelles/évolutives).



3. Conception statique

Réaliser un (ou plusieurs) diagramme(s) de classe pour le code. Représenter de préférence les classes du modèle dans un diagramme séparé.



Étape 2: Modélisation conceptuelle et logique de la base

ArtConnect

1. Identifier les entités et les relations

Entité	Attributs
Artist	name, bio, birthyear, disciplines, contactEmail, phone, city, website, socialMedia, isActive, artworks
Artwork	title, creationYear, type, medium, dimensions, description, price, status, artist, tags
ArtworkTag	name
Booking	workshop, member, bookingDate, paymentStatus
CommunityMember	name, email, birthyear, phone, city, favoriteDisciplines, membershipType, bookings, reviews
Discipline	name
Exhibition	title, startDate, endDate, description, gallery, curatorName, theme, artworks
Gallery	name, address, ownerName, openingHours, contactPhone, rating, website, exhibitions
Review	reviewer, artwork, rating, comment, reviewDate
Workshop	title, date, durationMinutes, maxParticipants, price, instructor, location, description, level

- Dépendances Fonctionnelles :

Artist : contactEmail -> name, bio, birthyear, phone, city, website, socialMedia, isActive.

Artwork : title, artist -> creationYear, type, medium, dimensions, description, price, status.

ArtworkTag : name ->

Booking : workshop, member, bookingDate -> paymentStatus.

CommunityMember : email -> name, birthyear, phone, city, membershipType.

Discipline : name ->

Exhibition : title, Gallery, startDate -> endDate, description, curatorName, theme.

Gallery : name, address -> ownerName, openingHours, contactPhone, rating, website.

Review : reviewer, artwork, reviewDate -> rating, comment.

Workshop : title, date, instructor -> durationMinutes, maxParticipants, price, location, description, level.

- **Identification des Identifiants (Clés Primaires) :**

Artist : id_artist

Artwork : id_artwork

Gallery : id_gallery.

Exhibition : id_exhibition.

Workshop : id_workshop.

CommunityMember : id_member.

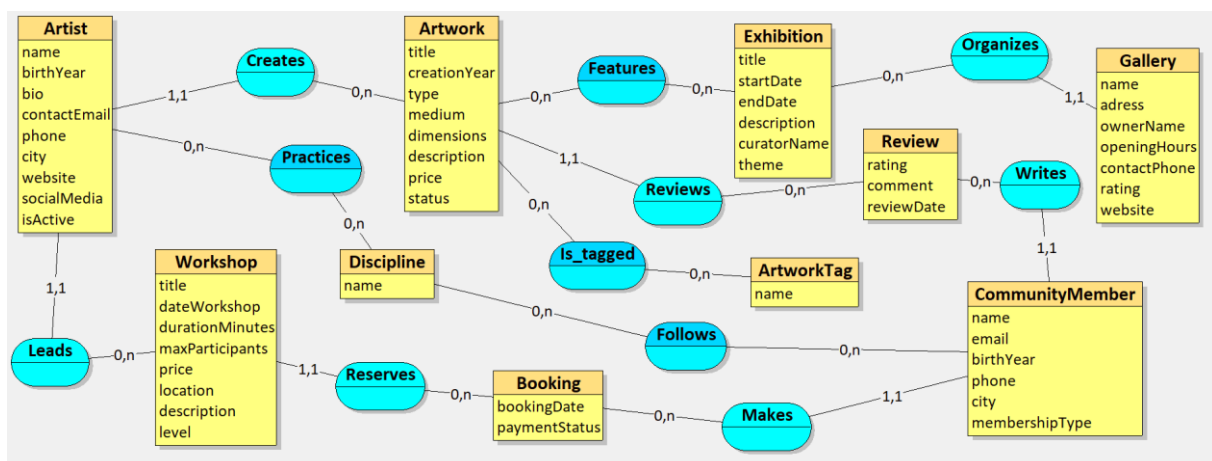
Booking : id_booking.

Review : id_review.

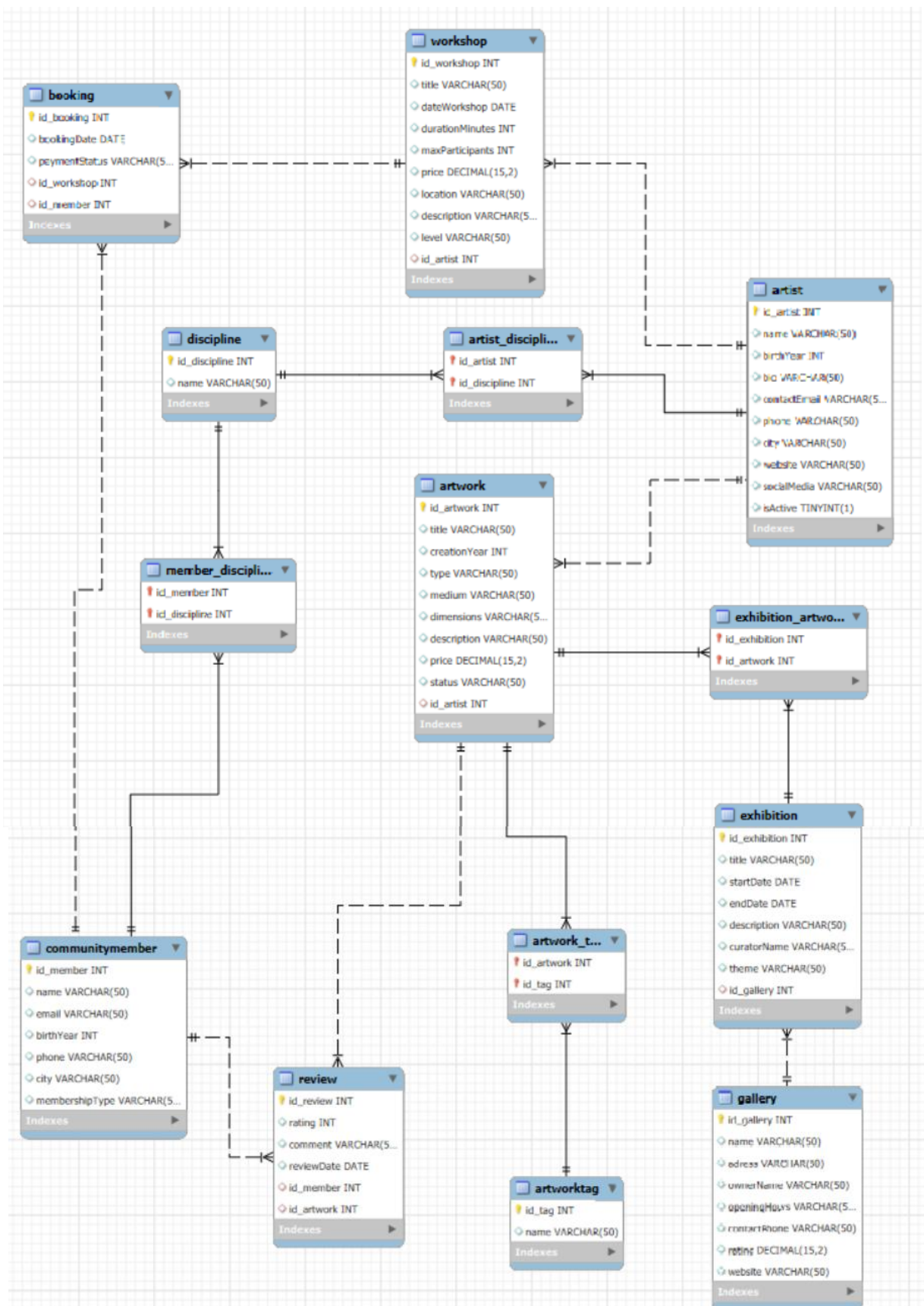
Discipline : id_discipline.

ArtworkTag : id_tag.

2. Créer un modèle conceptuel des données (MCD)



3. Créer un modèle logique des données (MLD)



Ici, dans ce modèle MLD, la base de données est déjà en 3FN, voici plusieurs explication :

- **Atomicité** : Toutes les valeurs ont été décomposés en éléments simple.
- **Identifiants** : Toutes les tables sont composés d'un clés primaires unique.
- **Cas de l'adresse dans Gallery** : elle a été conservé tel quel avec le type String en bloc même si on pouvait également le décomposer car il est traité comme une unité d'information indivisible pour l'affichage et cela facilitera le projet.
- **Dépendances fonctionnelles** : Chaque attribut ne dépend que de la clé primaire et il n'y a aucune dépendance transitive.

Étape 3 : Implémentation de la Base de Données et Fonctionnalités Avancées

Prompt utilisé pour le LLM pour insérer des données d'exemple :

Génère un script SQL pour l'insertion de données d'exemple pour la base ArtConnect. Créez un script d'insertion de données d'exemple (artistes, œuvres, événements, membres...) permettant de montrer des cas intéressants (plusieurs artistes, plusieurs événements, participations croisées, etc.). Contraintes :

- Ne pas réutiliser les noms d'artistes, d'œuvres, de galeries, d'expositions, de workshops ou de membres déjà présents dans les données de démonstration fournies par ArtConnect.
- Insérer plusieurs artistes, œuvres, galeries, expositions, ateliers et membres.
- Toutes les insertions doivent être en anglais

Documentation déclencheurs (trigger) :

TRIGGER 1 : trg_booking_check_capacity

S'active avant l'insertion d'une réservation. Ce trigger permet d'empêcher de dépasser la capacité maximale d'un atelier.

TRIGGER 2 : trg_exhibition_date_check

S'active avant l'insertion et mise à jour d'une exposition. Ce trigger permet de garantir que la date de fin est toujours postérieure à la date de début.

TRIGGER 3 : trg_review_rating_range

S'active avant l'insertion d'un avis. Ce trigger permet de contraindre la note entre 1 et 5 étoiles.

Documentation procédures stockées :

PROCEDURE 1 : create_workshop

Cette procédure permet de créer un workshop.

PROCEDURE 2 : get_artists_by_city

Cette procédure permet d'obtenir les artistes par ville.

PROCEDURE 3 : sp_register_for_art_path

Cette procédure permet qu'un membre effectue 2 réservations

Documentation vues :

VUE 1 : v_artist_overview

Vue de synthèse publique sur les artistes actifs en masquant les données sensibles pour une question de sécurité.

VUE 2 : v_artwork_catalog

Catalogue complet des œuvres avec artiste et tags.

VUE 3 : v_exhibition_program

Programme des expositions avec galerie et oeuvres exposés

VUE 4 : v_workshop_availability

Ateliers avec places restantes calculées en temps réel

VUE 5 : v_member_activity

Résumé de l'activité de chaque membre avec le nb ateliers suivis, nb avis, montant total dépensé.

Script de test des transactions :

```
-- =====
-- TÂCHE 5 - TRANSACTIONS
-- =====

-- Scénario : Un membre s'inscrit à un parcours
-- composé de deux ateliers. L'opération doit être atomique.

DELIMITER //

-- Cette procédure permet qu'un membre effectue 2 réservations
CREATE PROCEDURE sp_register_for_art_path(
    IN p_member_id INT,
    IN p_workshop1_id INT,
    IN p_workshop2_id INT
)
BEGIN
    -- Déclaration d'un gestionnaire d'erreur pour annuler la transaction en cas de problème
    -- (Par exemple si un trigger lève une exception "Atelier complet")
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        -- En cas d'erreur, on annule tout
        ROLLBACK;
        SELECT 'ERREUR : Échec de l'inscription au parcours. L'un des ateliers est peut-être complet ou inexistant. Aucune réservation n'a été effectuée.' AS Message;
    END;

    -- Début de la transaction
    START TRANSACTION;

    -- 1. Première réservation (Urban Color Lab)
    INSERT INTO Booking (id_booking, bookingDate, paymentStatus, id_workshop, id_member)
    VALUES ((SELECT COALESCE(MAX(id_booking), 0) + 1 FROM Booking b), CURDATE(), 'PENDING', p_workshop1_id, p_member_id);

    -- 2. Deuxième réservation (Industrial Photo Walk)
    INSERT INTO Booking (id_booking, bookingDate, paymentStatus, id_workshop, id_member)
    VALUES ((SELECT COALESCE(MAX(id_booking), 0) + 1 FROM Booking b), CURDATE(), 'PENDING', p_workshop2_id, p_member_id);

    -- Si tout s'est bien passé, on valide les changements
    COMMIT;

    SELECT 'SUCCÈS : Le membre a été inscrit avec succès aux deux ateliers du parcours.' AS Message;
END //

DELIMITER ;

-- =====
-- TEST DE LA TRANSACTION
-- =====

-- Test 1 : Inscription réussie au parcours (Ateliers 1 et 2 pour le membre 2)
CALL sp_register_for_art_path(2, 1, 2);

-- Vérification : le membre a bien 2 réservations en plus.
SELECT * FROM v_member_activity WHERE id_member = 2;
```

Étape 4: Intégration de la base dans l'application Java ArtConnect

Description de l'architecture retenue :

Le projet adopte une structure en 4 couches pour séparer les responsabilités :

1. **Présentation (UI)** : Interfaces graphiques JavaFX (fichiers FXML) et leurs Controllers qui gèrent les clics et l'affichage.
2. **Service** : Centralise la logique applicative et fait le pont entre l'interface et les données via le ServiceProvider.
3. **Accès aux Données (DAO/Persistence)** : Gère les requêtes SQL via JDBC pour communiquer avec la base de données MySQL.
4. **Modèle (Entities)** : Objets Java simples représentant les données (Artistes, Œuvres, etc.).

Flux de données : Utilisateur → Controller → Service → DAO (JDBC) → MySQL